

Remote Sensing GenAI



Aim

Synthetic Aperture Radar (SAR) images are essential for remote sensing applications. However, interpreting SAR images is often challenging due to their speckle noise (grainy texture) and grayscale format, which makes it difficult to analyze and extract critical information. Enhancing these images through colorization improves their interpretability and provides valuable insights for applications such as environmental monitoring, disaster management, agriculture, defense, and more.

Our Goal

We aim to develop a mobile application powered by Generative AI (GenAI) techniques to address the following challenges:

1. **Colorization of SAR Images:** Transform grayscale SAR images into vivid, colored visuals for enhanced readability and usability.
2. **Flood Area Detection:** Accurately identify and map regions affected by flooding to assist in disaster management and recovery efforts.
3. **Crop Image Classification:** Classify SAR crop images and match them to corresponding crop types on the ground using advanced GenAI models.

This innovative app bridges the gap between complex SAR data and practical real-world applications, enabling users to make data-driven decisions efficiently.

Table of Content

Model	Purpose
Pix2Pix CGAN	SAR Image Colorization
Unetr	SAR Image Segmentation for Flood detection
VIT	Crop Classification

Who Will Benefit/Use This App, and How? (USECASES)

Disaster Response Teams

The flood area detection feature automatically identifies flood affected on SAR images, eliminating manual interpretation. Teams can use these insights for efficient evacuation and resource allocation.

Environmental Researchers

The colorized SAR images provide clearer visuals of land changes, deforestation, or water body dynamics, enabling data-driven research and decision-making.

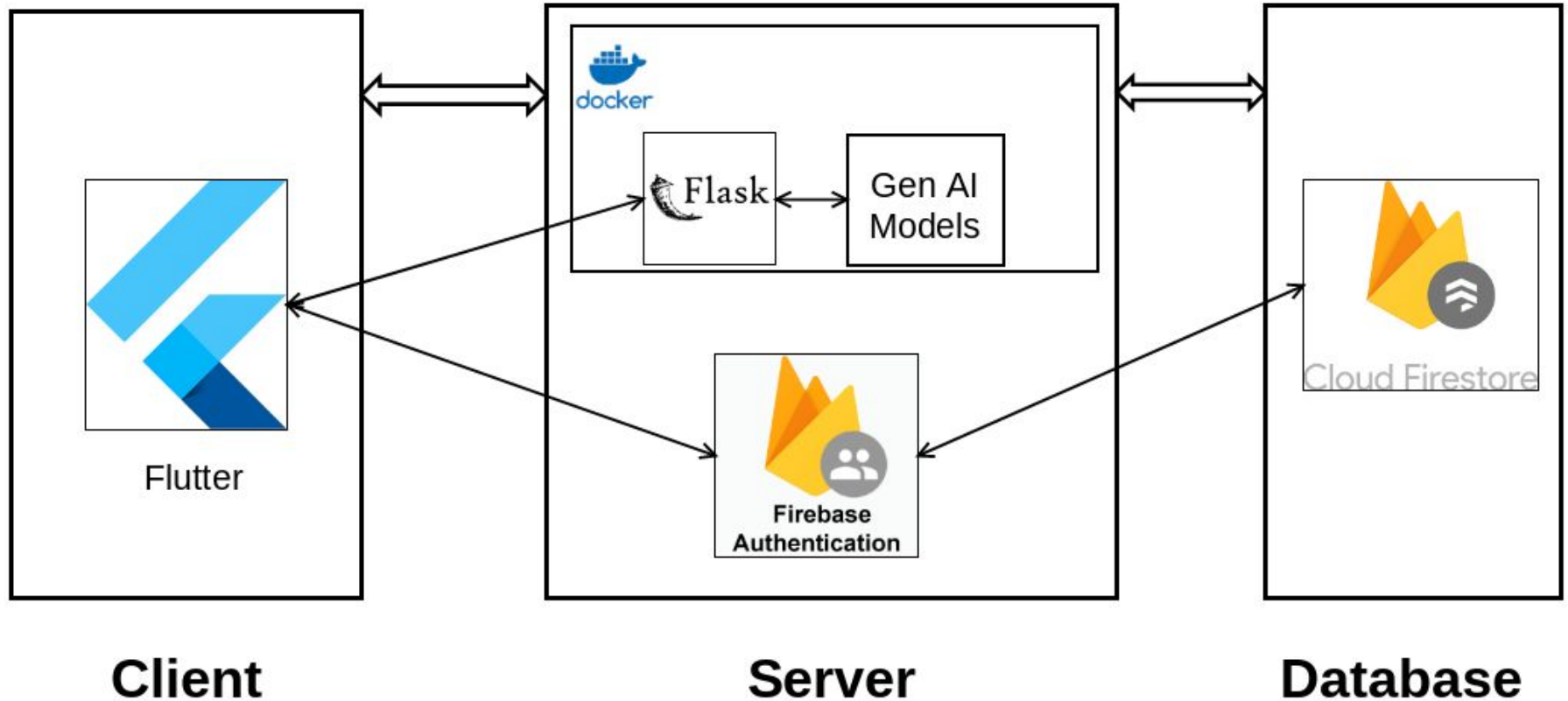
Farmers and Agricultural Agencies

The app highlights areas affected by floods or waterlogging, helping farmers make better decisions for crop planning and resource management.

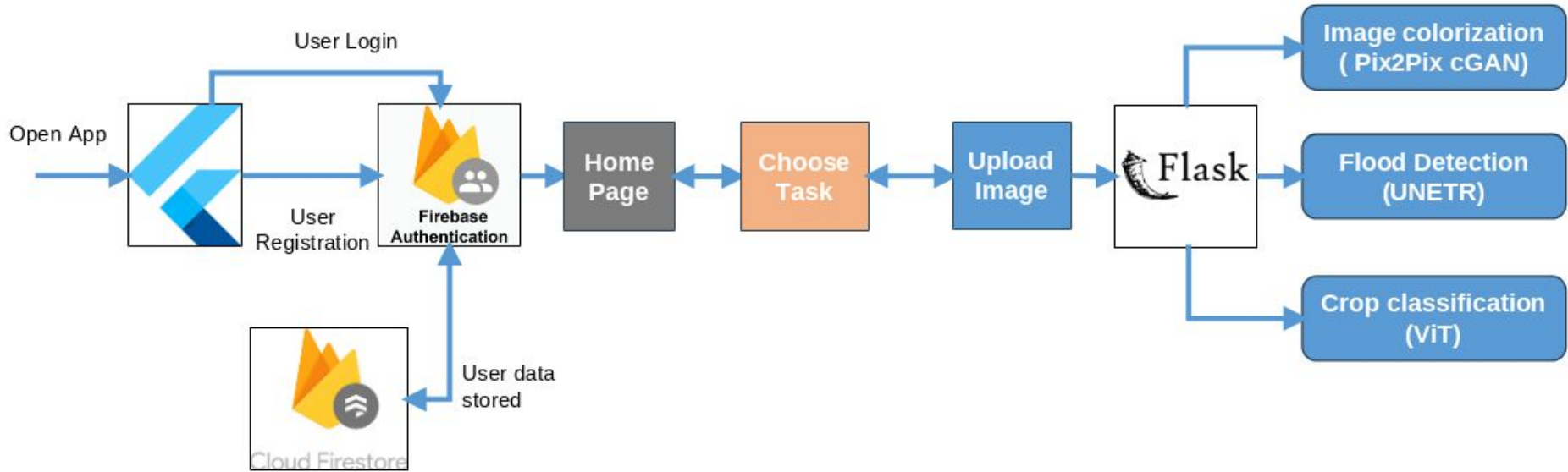
Defense Agencies

The enhanced interpretability of SAR images aids in surveillance and decision-making in challenging terrains and weather conditions.

Architecture Diagram and Tech Stack



Workflow



Datasets Used

1. Sentinel-1 & 2 Dataset for Colorization

- Total Image Pairs: 16,000 SAR (Sentinel-1) & optical (Sentinel-2) image pairs
- Geographic Areas: Agricultural, grasslands, urban, barren
- Split: 80% train (12,800), 20% validation (3,200)
- Link: <https://www.kaggle.com/datasets/requiemonk/sentinel12-image-pairs-segregated-by-terrain>

2. Flood Detection Dataset

- Train: 1120
- Valid: 140
- Test: 140
- Link : <https://www.kaggle.com/code/samvram/flood-detection-sar>

3. Crop Classification Dataset

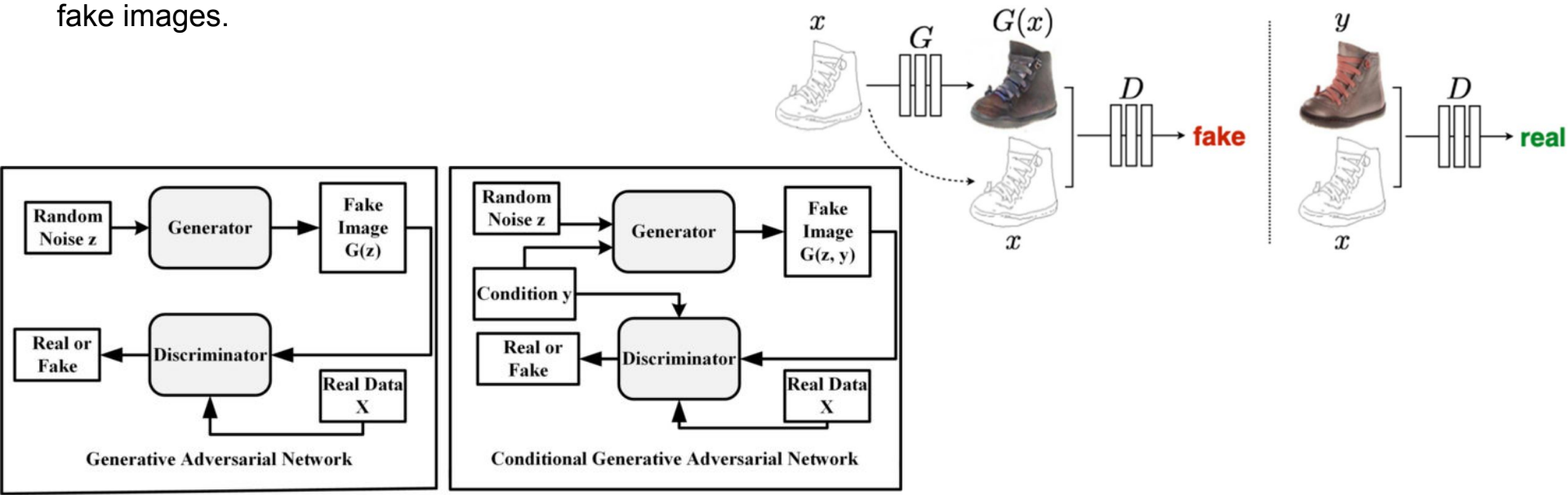
- Classes: 5 crop categories (e.g., wheat, rice, etc.)
- Training: 40 images per class (200 total)
- Validation: 10 images per class (50 total)
- Link: <https://www.kaggle.com/datasets/aman2000jaiswal/agriculture-crop-images>

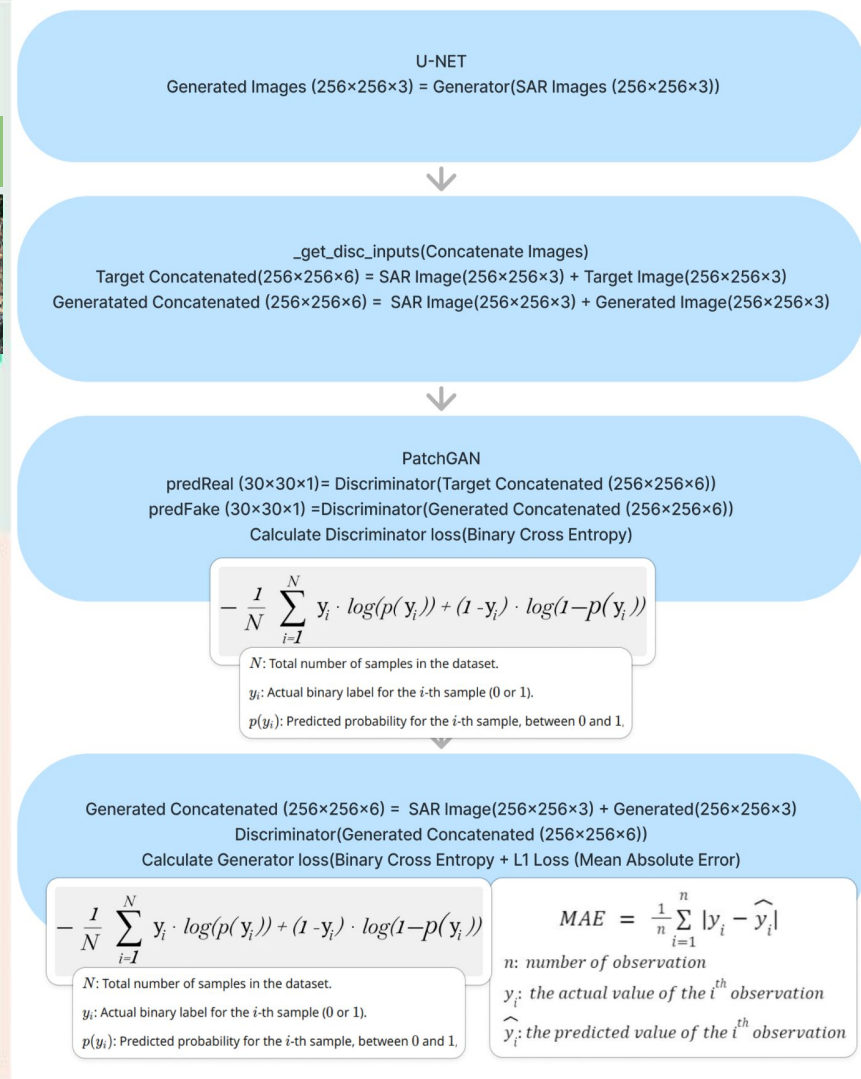
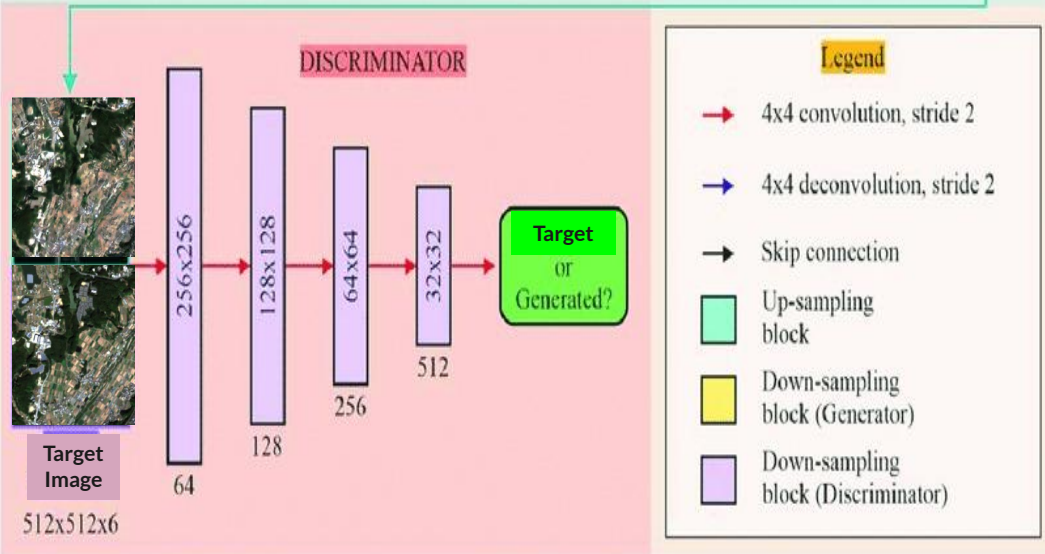
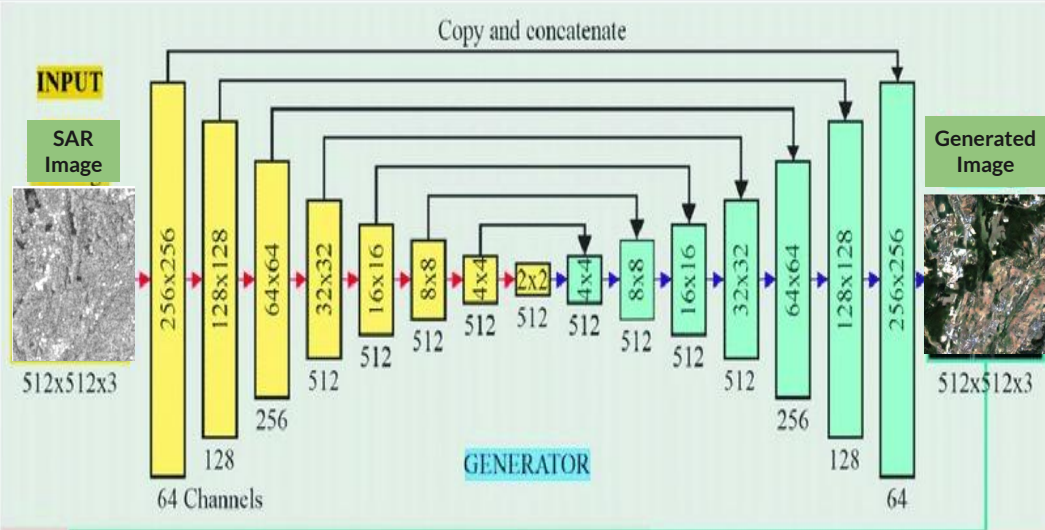
Pix2Pix CGAN for SAR Image Colourization

Goal: Learn to convert input images (e.g., sketches, maps) into output images (e.g., realistic photos) using a **conditional GAN**.

- **Generator:** Takes an input image and generates a corresponding output image (e.g., turns a sketch into a photo).
- **Discriminator:** Distinguishes between **real** (from dataset) and **fake** (generated) image pairs.

Training: The generator tries to **fool** the discriminator, while the discriminator learns to **classify** real vs. fake images.





Evaluation : Fréchet Inception Distance(FID) score

FID evaluates the quality of images generated by models like Pix2Pix.

- How it works: Compares the feature distributions of real and generated images using a pre-trained Inception model.
- Metric: Calculates the Frechet distance between the means and covariances of these feature distributions.
- Interpretation: Lower FID scores indicate better quality (more realistic) generated images.

$$FID = \|\mu_r - \mu_g\|^2 + \text{Tr} \left(\Sigma_r + \Sigma_g - 2 \cdot (\Sigma_r \Sigma_g)^{\frac{1}{2}} \right)$$

Where:

- μ_r : Mean vector of the real distribution (d -dimensional)
- μ_g : Mean vector of the generated distribution (d -dimensional)
- Σ_r : Covariance matrix of the real distribution ($d \times d$)
- Σ_g : Covariance matrix of the generated distribution ($d \times d$)
- Tr : Trace of a matrix (sum of diagonal elements)

Our model achieved the following FID score:

FID Score: 320.46373

UNETR for Flood Detection

WHAT IS UNETR?

- UNETR (U-Net with Transformer Encoder) is an advanced neural network architecture for semantic segmentation tasks.
- Combines transformer encoders with U-Net for semantic segmentation.
- Captures global and local features simultaneously.

WHY USE UNETR FOR FLOOD DETECTION?

- Accurate segmentation: Identifies flooded vs. non-flooded areas pixel by pixel.
- Global understanding: Handles irregular flood patterns using transformers.
- SAR compatibility: Works well with noisy VV-polarized SAR images.
- Scalable: Ideal for high-resolution satellite imagery.

```
cf = {}
# cf["image_size"] = 512
cf["image_size"] = 256
cf["num_channels"] = 3
cf["num_layers"] = 12
cf["hidden_dim"] = 768
# cf["mlp_dim"] = 32
cf["mlp_dim"] = 1536
cf["num_heads"] = 6
cf["dropout_rate"] = 0.1
cf["patch_size"] = 16
cf["num_patches"] = (cf["image_size"]**2)//(cf["patch_size"]**2)
cf["flat_patches_shape"] = (
    cf["num_patches"],
    cf["patch_size"]*cf["patch_size"]*cf["num_channels"])
```

Linear Projection: Transforms input patches into feature vectors using a weight matrix and matrix multiplication.

Patch Embedding: Converts image patches into embeddings; **Positional Embedding** adds location information.

Normalization: Stabilizes training.

- **Layer Normalization:** Applied within the transformer encoder to stabilize training by normalizing inputs across features. Normalizes inputs across features. Helps maintain network stability during training.
- **Batch Normalization:** Used in the convolutional layers of the decoder to reduce internal covariate shift and improve convergence. Normalizes inputs across the batch dimension. Reduces internal covariate shift and improves network convergence.

Multi-Head Attention: Focuses on relevant input parts by computing weighted sums using scaled dot-product attention.

Multilayer Perceptron (MLP): Enhances features with two dense layers and dropout.

GELU vs. ReLU:

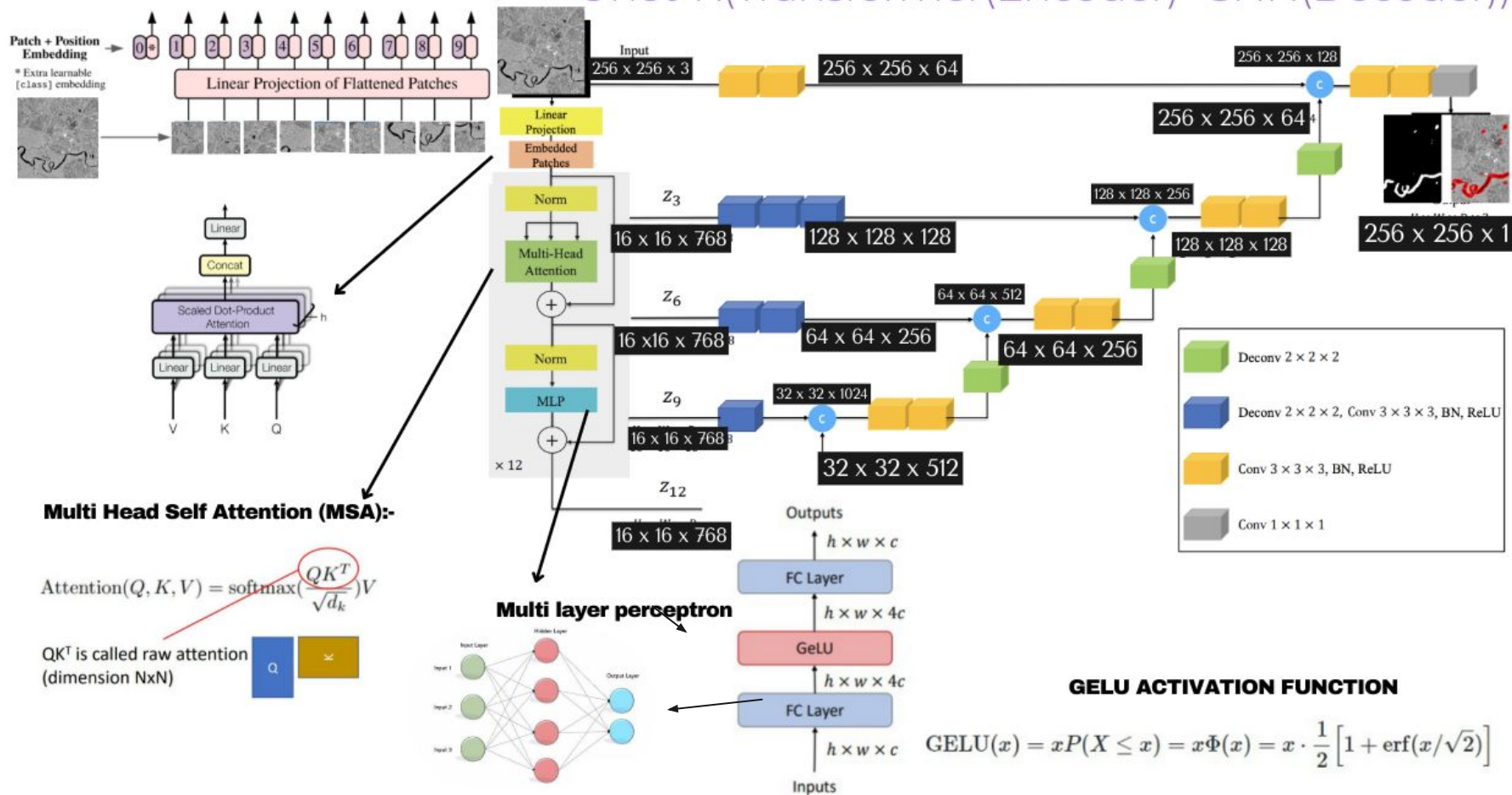
- GELU: Smooth activation; avoids dead neurons.
- ReLU: Sharp cutoff; better for localized features.

Dice Coefficient & Loss: Measures overlap; loss penalizes poor segmentation.

Learning Rate Decay: Reduces learning rate over time (e.g., cosine decay).

UNET-R FOR IMAGE SEGMENTATION

Unet-R(Transformer(Encoder)+CNN(Decoder))



Linear Projection:

Linear Projection:

$y = x \cdot W$, where $x \in \mathbb{R}^{H \times W \times C}$, $W \in \mathbb{R}^{\text{flattened size} \times \text{feature size}}$.

- x : Flattened input patch (vector of pixel values).

- W : Weight matrix (learned during training).

- y : Resulting feature vector.

Normalization:

- Layer Norm: $\text{Norm}(z_i) = \frac{z_i - \mu}{\sigma + \epsilon}$.

- Batch Norm: $\text{BN}(x) = \frac{x - \mathbb{E}[x]}{\sqrt{\text{Var}[x] + \epsilon}}$.

- z_i : Input feature.

- μ : Mean of the features.

- σ : Standard deviation of the features.

- ϵ : Small constant for numerical stability.

- x : Input feature across the batch.

- $\mathbb{E}[x]$: Mean of the batch.

- $\text{Var}[x]$: Variance of the batch.

- ϵ : Small constant for numerical stability.

Scaled Dot-Product Attention:

Attention Weights = $\text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{d_k}}\right)$.

Output = Weights $\cdot V$.

- Q : Query matrix.

- K : Key matrix.

- V : Value matrix.

- d_k : Dimensionality of the keys.

- softmax : Converts scores into probabilities.

- X : Predicted segmentation region.

- Y : True segmentation region.

- $|X \cap Y|$: Overlap (intersection) of predicted and true regions.

- $|X| + |Y|$: Total size (union) of predicted and true regions.

Dice Coefficient:

$$\text{Dice} = \frac{2 \cdot |X \cap Y|}{|X| + |Y|}$$

Gelu:

$$\text{GELU}(x) = xP(X \leq x) = x\Phi(x) = x \cdot \frac{1}{2} \left[1 + \text{erf}(x/\sqrt{2}) \right]$$

- x : The input value.
- $P(X \leq x)$: The probability that a random variable X from a standard normal distribution is less than or equal to x . This is equivalent to the cumulative distribution function $\Phi(x)$ of the standard normal distribution.
- $\Phi(x)$: Cumulative Distribution function calculates the area under the curve of the normal distribution from $-\infty$ to x .

Tversky Loss

$$\text{Tversky}(P, G) = \frac{|P \cap G|}{|P \cap G| + \alpha \cdot |P \setminus G| + \beta \cdot |G \setminus P|}$$

Where:

- P is the predicted set,
- G is the ground truth set,
- α and β are parameters that control the balance between false positives and false negatives.

The Tversky Loss is then:

$$\text{Loss}_{\text{Tversky}} = 1 - \text{Tversky}(P, G)$$

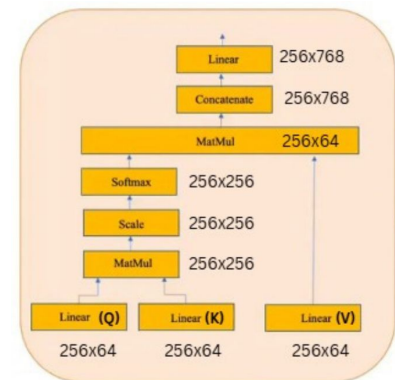
Evaluation Metrics

```
Test Loss: 0.5617552399635315
Test Dice Coefficient: 0.42420345544815063
Test IoU: 0.3229259252548218
Test Accuracy: 0.9862749576568604
Test Sensitivity: 0.4538205862045288
Test Precision: 0.5614367127418518
Test Specificity: 0.996077299118042
```

Multi Head Attention



L=12 blocks



CONCATENATION OF 12 HEADS :

Vision Transformer (ViT) for Crop Classification

Vision Transformer (ViT) is a deep learning model that applies Transformer architecture (originally for NLP) to image classification tasks.

- **Image Preprocessing**:- The image is divided into fixed-size patches (e.g., 16x16), each flattened into a vector.
- **Embedding**:- Each patch is linearly embedded into a fixed-dimensional vector.
- **Positional Encoding**:- Positional embeddings are added to maintain spatial information.
- **Transformer Encoder**:- Patch embeddings are processed by Transformer layers using self-attention to capture dependencies.
- **Classification**:- The [CLS] token's output from the Transformer is used for classification.
- **Loss Function**:- **Cross Entropy Loss** compares predicted class probabilities (from Softmax) to true labels.

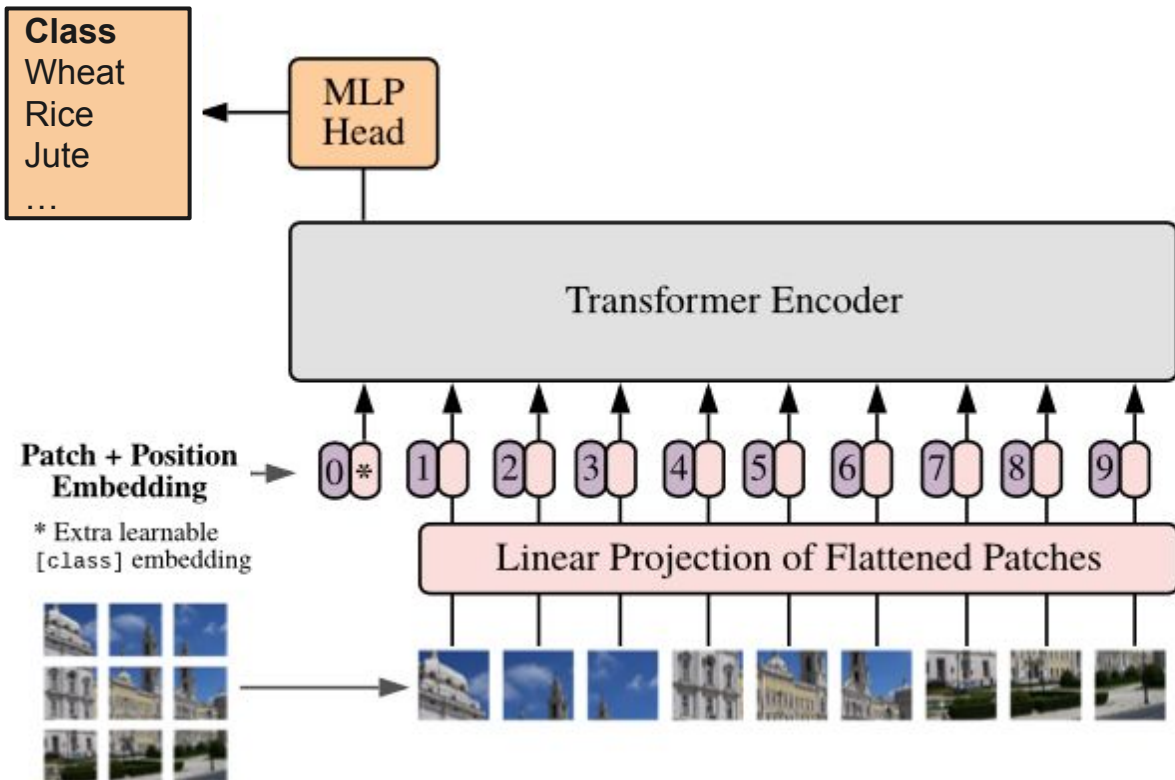
$$\text{Loss} = - \sum_{j=1}^K y_j \log(\hat{y}_j)$$

where k is number of classes in the data

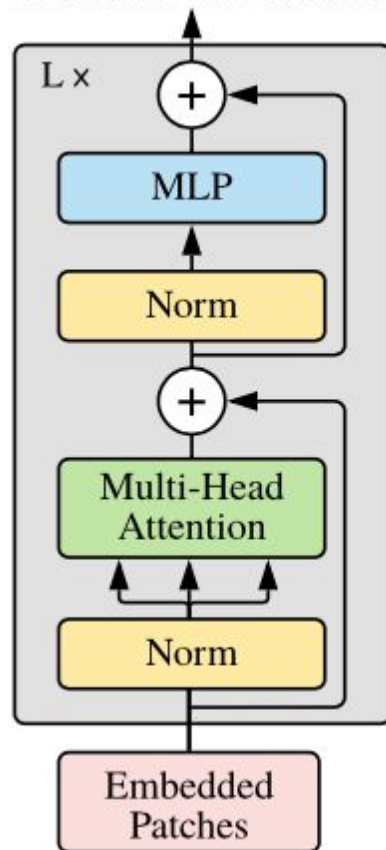
$\hat{y}_j$ — *predicted value*

y_j — *true value*

Vision Transformer (ViT)



Transformer Encoder



Thank You